

CAREER ROADMAP

From Lily to Game Engine Engineer

A 60-week, C-first study sequence for Linux • designed for 10–12 focused hours per week

Target outcome. By week 60, you will have a small but coherent engine, a playable 2.5D demo, and portfolio evidence that you can reason about memory, rendering, physics, assets, tools, and performance—not merely use an existing engine.

START HERE

Lily already has memory, containers, and file I/O. Keep them. Use the first month to audit interfaces, tests, failure behavior, and profiling hooks; then move forward to platform timing and math.

The operating system for your learning

65 / 25 / 10

Build / read / explain. Most learning should end in running code.

One growing engine

Lily is the laboratory; small sandboxes isolate risky ideas first.

Evidence over features

Every phase ends with tests, a profiler capture, and a tiny demo.

Just-in-time theory

Read the chapter required by the next implementation—not entire shelves.

The sequence at a glance

PHASE	WEEKS	CORE FOCUS	EXIT PROOF
0	1–4	C + systems audit	Clean diagnostics, tests, benchmarks
1	5–10	Math + platform loop	Deterministic loop, input, transforms
2	11–18	CPU graphics pipeline	Textured, depth-tested mesh rasterizer
3	19–30	GPU renderer	2.5D scene with batching and lights
4	31–40	Assets + architecture	Cooked assets, handles, hot reload
5	41–50	Collision + physics	Stable 2D contacts at fixed timestep
6	51–60	Capstone + portfolio	Playable slice, profiling report, docs

Pace adjustment: at 6 hours/week, treat this as a 24–30 month plan; at 18–20 hours/week, 9–12 months is possible. Do not compress exit proofs.

MILESTONE PROJECTS

Turn each topic into an engineering proof

A phase is complete only when the artifact works, is tested, and can be explained from first principles.

0

C & systems audit
weeks 1–4

LEARN Object lifetime, undefined behavior, alignment, compilation/linking, cache behavior, sanitizers, perf and call stacks.

BUILD Harden Lily's existing memory, containers and file I/O; add failure-path tests, allocation diagnostics and microbenchmarks.

DONE WHEN Zero sanitizer findings in the test suite; benchmark report names the bottleneck and measurement method.

1

Math & platform
weeks 5–10

LEARN Vectors, matrices, transforms, coordinate spaces, interpolation, quaternions; clocks, events, input and virtual memory.

BUILD Implement lily_math plus a Linux platform loop with fixed update, variable render, input mapping and frame timing.

DONE WHEN Transform tests pass; replayed inputs produce the same simulation state; frame-time spikes are visible.

2

CPU renderer
weeks 11–18

LEARN Model/view/projection, clipping, barycentric coordinates, rasterization, depth, texture sampling and color spaces.

BUILD Build a framebuffer renderer: lines → triangles → clipping → z-buffer → textured mesh → simple directional light.

DONE WHEN A rotating textured scene renders correctly; golden-image tests and a profile identify hot loops.

3

GPU renderer
weeks 19–30

LEARN OpenGL/GLSL pipeline, buffers, textures, shaders, state changes, batching, render passes, transparency and shadows.

BUILD Implement OpenGL first behind Lily's render interface. Add sprites, tile layers, mesh materials, camera and debug drawing.

DONE WHEN A 2.5D scene holds the frame budget under load; draw calls, GPU time and overdraw are documented. Vulkan comes later.

4

Assets & engine
architecture
weeks 31–40

LEARN Stable handles, ownership, dependency graphs, serialization, asset cooking, hot reload, world storage and job boundaries.

BUILD Add VFS/asset registry, offline cooker, versioned formats, resource states, scene loading and a minimal inspector/debug UI.

DONE WHEN Missing/corrupt assets fail safely; hot reload preserves valid state; a dependency diagram matches the code.

5

Collision & physics
weeks 41–50

LEARN Fixed timestep, integration, AABB/circle/SAT tests, broad phase, manifolds, impulses, friction, sleeping and CCD basics.

BUILD Create a 2D physics sandbox, then integrate only the features needed by Lily's top-down 2.5D game.

DONE WHEN Stacks settle without exploding; 1,000 simple colliders update at 60 Hz on your target machine; replay is stable.

6

Capstone vertical
slice
weeks 51–60

LEARN Integration, tooling, performance budgets, packaging, crash diagnosis, technical writing and design trade-offs.

BUILD Ship one polished room: streamed assets, animated character, collision, interactions, enemies, UI, save/load and audio hooks.

DONE WHEN A new machine can build and run it from documented steps; publish a 3–5 minute demo and two profiling case studies.

REFERENCES & EXECUTION

A small bookshelf, used at the right moment

Books are inputs. The portfolio artifacts on page 2 are the output.

WHEN	PRIMARY	USE IT FOR
NOW	Effective C, 2e Computer Systems: A Programmer's Perspective, 3e 3D Math Primer	Professional C habits; selected chapters on machine code, linking, memory hierarchy and virtual memory; then vectors, matrices and coordinate spaces.
GRAPHICS	LearnOpenGL Foundations of Game Engine Development, Vol. 2 Real-Time Rendering, 4e	Follow one tutorial path while implementing in C; use the two books as explanations and desk references. Read PBRT only for deeper light transport later.
ENGINE DESIGN	Game Engine Architecture Data-Oriented Design	Subsystem boundaries, asset pipeline, runtime loop, tooling and performance. Translate the C++ examples into C concepts; do not copy class hierarchies.
PHYSICS	Game Physics Engine Development, 2e Real-Time Collision Detection Gaffer on Games: Physics	Build the simple solver from first principles; use Ericson for collision queries and robustness; use Fiedler for integration and fixed-timestep intuition.
LATER	PBRT, 4e (free online) Khronos Vulkan docs	Advanced rendering and explicit GPU control after the OpenGL renderer is stable. Vulkan is a second renderer backend, not your first graphics lesson.

A sustainable 10–12 hour week

MON 90 min	TUE 2 hr	THU 2 hr	SAT 4 hr	SUN 60 min
Theory + handwritten derivation	Implement one narrow feature	Tests, debugger, profiling	Integration + visible milestone	Write notes + choose next task

Your next 30 days in Lily

WEEK 1 Engine loop	WEEK 2 Math core	WEEK 3 Input + replay	WEEK 4 First pixels
Monotonic clock, fixed update, variable render, frame-time log.	vec2/3/4, mat3/4, transforms and tests for conventions.	Action mapping, recorded input stream, deterministic state hash.	CPU framebuffer, clear, line, rectangle and screenshot test.
SCOPE GUARD		Do not begin Vulkan, a general ECS, multithreaded jobs, or 3D rigid-body physics until a measured milestone needs them. Assembly belongs only after profiling identifies a hot path and the compiler output has been inspected.	

Graduation evidence CPU rasterizer • GPU renderer • physics sandbox • playable Lily slice •
reproducible build • two profiling case studies • concise architecture document